PROJECT REPORT

BSc Computer Science (2nd Year)

Module: CST2550 Software Engineering Management and Development

Group Coursework: Hospital Outpatient Booking System

Group Name: Mocha Tech Team

TEAM DETAILS

Name: Keyon Bent (M00867462)
Role: Team leader

Name: Mohamed Jama (M00919644)
Role: Secretary

Name: Zarsham Ali Haider (M00733239)
Role: Tester

Name: Gabriel Mendes (M00966597)
Role: Developer

Name: Ismail Ahmed (M00938887)
Role: Developer

Name: Costin Ciuplea (M00984643)
Role: Developer

Table of Contents

I. Introduction 1-2

II. Design 3-4

III. Testing 5-6

IV. Conclusion 7

V. References 8

Introduction

The project was developed using Agile project management principles, with the Scrum framework used to organise the team's development process. Agile encourages iterative development, continuous feedback, and collaboration throughout the software development lifecycle. Scrum teams typically consist of defined roles such as the Scrum Master and the development team. The Scrum Master facilitates the process and ensures that Agile practices are followed, while the development team is responsible for implementing system functionality (Atlassian, n.d.; Coursera, 2023).

In this project, the roles were adapted to suit the team structure and included a team leader (Scrum Master), developers, a tester, and a secretary. Each role contributed to the development of the system through collaboration, regular meetings such as stand-ups, and ongoing feedback between team members.

This project focuses on the design and implementation of a hospital outpatient booking system intended to improve the efficiency of scheduling patient appointments. Traditional outpatient booking methods, such as telephone-based or manual booking systems, often lead to long waiting times, inefficient allocation of resources, and increased administrative workload. These limitations can negatively affect both patient satisfaction and the quality of healthcare service delivery (Samadbeik et al., 2018).

With the continued development of digital healthcare technologies, online appointment booking systems have become an increasingly important solution for modern healthcare environments. Such systems allow patients to book, update, or cancel appointments more conveniently without relying on manual processes. Previous studies have shown that digital appointment systems can improve patient satisfaction, increase operational efficiency, and optimise the use of healthcare resources (Betancor et al., 2025; Atherton et al., 2024).

The aim of this project is to design and implement an outpatient booking system that efficiently manages user and appointment data using appropriate data structures and algorithms. The system focuses on ensuring reliable booking operations, preventing appointment conflicts, and providing clear information regarding appointment availability.

This report is structured as follows. The Design section explains the selected data structures and algorithms, including justification and pseudocode. The Testing section outlines the testing approach and presents the test cases used to validate the system. Finally, the Conclusion summarises the work completed, reflects on the development process, and suggests possible improvements for future work.

Design

2.1 System Architecture

The system was designed using a modular backend architecture in order to separate responsibilities between different components of the application. Modular design improves maintainability and readability because each part of the system performs a specific task. Separating responsibilities in this way is a common practice in software engineering because it helps reduce complexity and makes systems easier to update or extend in the future (Pressman and Maxim, 2020).

The backend system contains three primary components:

- **AuthController** – responsible for user registration and authentication
- **AppointmentController** – responsible for handling appointment-related requests
- **DatabaseService** – responsible for database communication and data operations

The controllers validate user input and process requests, while the DatabaseService performs the underlying data operations. This layered structure keeps the system organised and prevents business logic from being mixed with database operations.

2.2 Data Structures Used

Several data structures were used within the system depending on the type of information being stored and how frequently the data needs to be accessed or updated.

Array – Appointment Time Slots

A fixed array is used to store the available appointment times within a single day.

Example structure:

AppointmentTimes = [09:00, 10:00, 11:00, 12:00, 13:00, 14:00, 15:00, 16:00, 17:00]

An array is appropriate here because the number of appointment slots available per day is fixed and does not change during system execution. Arrays allow fast indexed access to elements, meaning individual time slots can be accessed immediately without searching through the structure (Cormen et al., 2009).

Time complexity for accessing a time slot:

O(1)

List – Appointment Records

Lists are used to store collections of appointment records retrieved from the database. Lists are dynamic structures that allow elements to be added or removed during runtime, making them suitable for situations where the number of stored items may change over time.

Time complexity:

Search operation: O(n)
Insertion operation: O(1) (amortised)

Dictionary – Appointment Availability Status

A dictionary structure is used to store the booking status for each date. Each date acts as the key, while the status value indicates whether the day is available, limited, or fully booked.

Example structure:

date → status

Possible status values include:

- available
- limited
- none

Dictionaries are efficient because they allow quick retrieval of values using keys. This allows the system to quickly check the availability status of a particular date (Goodrich, Tamassia and Goldwasser, 2014).

Average lookup complexity:

O(1)

2.3 Appointment Booking Algorithm

The appointment booking algorithm ensures that a time slot cannot be booked more than once.

When a user attempts to create an appointment, the system first checks whether a booking already exists for the same date, time, and hospital. If a booking already exists, the system rejects the request. Otherwise, the appointment is created successfully.

Pseudocode – Appointment Creation

```
FUNCTION CreateAppointment(date, time, hospital)

SEARCH database for appointment with same date, time, hospital

IF appointment exists
   RETURN "Slot already booked"

ELSE
   INSERT appointment into database
   RETURN "Appointment created"

END FUNCTION
```

Time complexity:

Worst-case search complexity: **O(n)**
where *n* represents the number of stored appointments.

2.4 User Authentication Algorithm

The authentication component manages both registration and login functionality.

Pseudocode – User Registration

```
FUNCTION RegisterUser(userDetails)

IF required input fields are missing
   RETURN error

SEARCH database for existing user email

IF email exists
   RETURN "User already registered"

ELSE
   INSERT new user record
   RETURN "Registration successful"

END FUNCTION
```

Pseudocode – User Login

```
FUNCTION LoginUser(email, password)

SEARCH database for matching credentials

IF user found
   RETURN user details

ELSE
   RETURN "Invalid login"

END FUNCTION
```

Time complexity:

Database search complexity: **O(n)**
where *n* represents the number of registered users.

2.5 Appointment Availability Algorithm

The system also calculates the booking status of each day based on the number of appointment slots already booked.

Pseudocode – Daily Availability

```
FOR each day

   count booked appointments

   IF booked appointments == total slots
      status = "none"

   ELSE IF booked appointments >= threshold
      status = "limited"

   ELSE
      status = "available"

STORE status in dictionary

END FOR
```

Time complexity:

O(n), where *n* represents the number of appointments for that day.

Testing

3.1 Testing Strategy

Testing is an important part of the software development process because it helps ensure that the system behaves correctly and reduces the likelihood of defects. Unit testing focuses on verifying small components of the system individually to confirm that they function as expected (Hamill, 2004).

In this project, automated unit testing was used to check whether the main backend functions of the system worked as expected. The tests were implemented using MSTest, which is Microsoft's built-in testing framework for .NET applications.

The tests mainly focused on the DatabaseService class because it contains the core functionality responsible for user management and appointment booking.

3.2 Testing Environment

Component	Description
Development Environment	Microsoft Visual Studio
Programming Language	C# (.NET)
Testing Framework	MSTest
Database	SQL Server LocalDB
Testing Tool	Visual Studio Test Explorer

3.3 Unit Test Cases

Test ID	Test Case	Description	Expected Result
T1	RegisterUser_ThenUserExists_ReturnsTrue	Tests user registration	User stored successfully
T2	GetUserByCredentials_WithCorrectPassword_ReturnsUser	Tests valid login	User returned
T3	GetUserByCredentials_WithWrongPassword_ReturnsNull	Tests invalid login	Login rejected
T4	GetUserDetailsByEmail_ReturnsExpectedDetails	Retrieves stored user data	Details match input
T5	CreateAppointment_WithFreeSlot_ReturnsTrue	Books appointment	Appointment created
T6	CreateAppointment_WithDuplicateSlot_ReturnsFalse	Prevents double booking	Booking rejected
T7	AppointmentSlotExists_AfterCreate_ReturnsTrue	Checks slot availability	Slot detected
T8	GetBookedTimes_ReturnsBookedTimes	Retrieves booked times	Time appears in list
T9	UpdateAppointment_ChangesAppointmentDetails	Updates appointment	Details updated
T10	DeleteAppointment_RemovesAppointment	Deletes appointment	Appointment removed
T11	GetDayStatuses_AllSlotsBooked	Fully booked day	Status = none
T12	GetDayStatuses_PartiallyBooked	Limited availability	Status = limited
T13	GetDayStatuses_FewBookings	Available day	Status = available

3.4 Test Results

Total Tests	Passed	Failed
14	14	0

All tests passed successfully, indicating that the core backend functionality operates correctly.

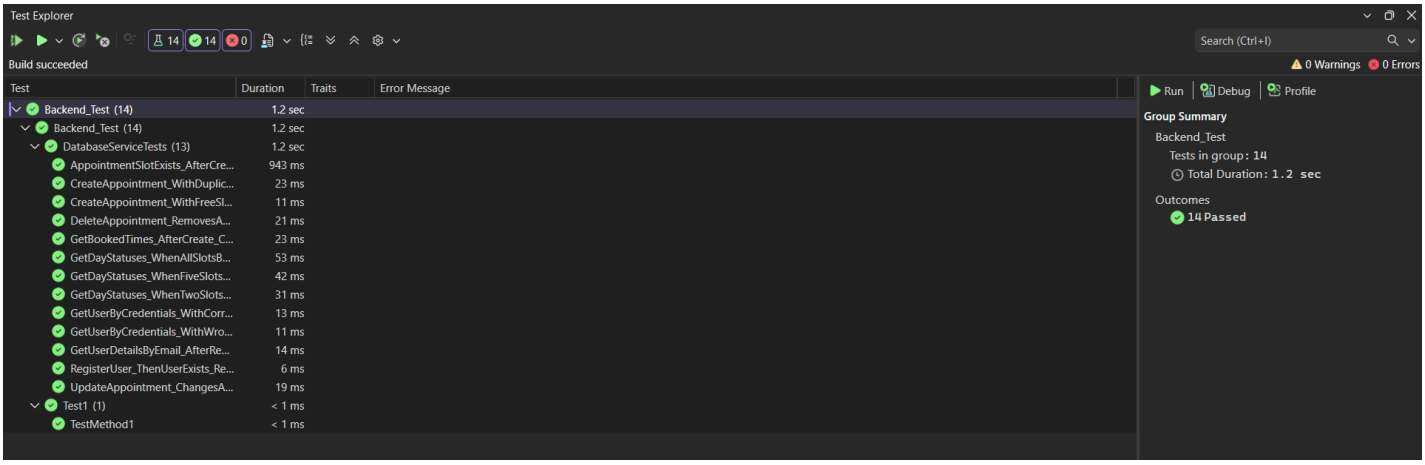

Figure 1: Unit testing results shown in Visual Studio Test Explorer.

Conclusion

This project successfully designed and implemented a hospital outpatient booking system intended to improve the process of scheduling patient appointments. The system was developed using Agile principles and the Scrum framework, which supported collaboration, iterative development, and continuous feedback among team members.

The final system allows users to register accounts, log into the platform, and manage outpatient appointments. Patients can create, update, and cancel appointments, while the system prevents the same appointment slot from being booked more than once. The application also provides clear appointment availability information by calculating the booking status for each day.

The backend architecture was organised using controllers and service classes to separate responsibilities within the system. Appropriate data structures such as arrays, lists, and dictionaries were used to manage appointment data and availability information. In addition, unit testing was implemented using the MSTest framework to verify key system functionality. The automated tests confirmed that registration, login authentication, appointment booking, updating, and deletion all operated correctly.

During the development process, the team gained practical experience in applying software engineering concepts such as Agile development, system design, and automated testing. This helped the team to better understand how different stages of the software development lifecycle connect in a real project.

Although the system meets the main requirements outlined in the project brief, several limitations remain. One reason for these limitations was the restricted time available for development, which meant the team had to focus mainly on the core backend functionality rather than more advanced features. As a result, the system does not currently include role-based access control, automated appointment reminders, or integration with external healthcare systems. In addition, testing focused mainly on backend unit testing, so broader integration and performance testing were outside the scope of the project.

If the project were to be completed again, the team would improve its approach by planning more time for advanced functionality, broader testing, and earlier refinement of the report. It would also be beneficial to improve security by introducing password hashing and stronger authentication methods, and to improve usability through a more polished user interface. Further optimisation could also be carried out to support larger datasets more efficiently.

Overall, the project demonstrates how software engineering principles and structured development practices can be used to create a functional outpatient booking system that improves appointment management within a healthcare environment. It also provided valuable experience in teamwork, planning, implementation, and testing, while highlighting areas that could be improved in future development work.

References

Atlassian (n.d.) ‘What is Scrum? A guide to the Agile framework’. Available at: https://www.atlassian.com/agile/scrum (Accessed: 9 April 2026).

Atherton, H., Eccles, A., Poltawski, L., Dale, J., Campbell, J. and Abel, G. (2024) ‘Investigating patient use and experience of online appointment booking in primary care: mixed methods study’, Journal of Medical Internet Research, 26, e51931. doi: 10.2196/51931.

Betancor, P.K., Boehringer, D., Jordan, J., Lüchtenberg, C., Lambeck, M., Ketterer, M.C., Reinhard, T. and Reich, M. (2025) ‘Efficient patient care in the digital age: impact of online appointment scheduling in a medical practice and a university hospital on the “no-show”-rate’, Frontiers in Digital Health, 7, 1567397. doi: 10.3389/fdgth.2025.1567397.

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009) Introduction to Algorithms. 3rd edn. Cambridge, MA: MIT Press.

Coursera (2023) ‘The 3 Scrum roles and responsibilities, explained’. Available at: https://www.coursera.org/gb/articles/scrum-roles-and-responsibilities (Accessed: 9 April 2026).

Goodrich, M.T., Tamassia, R. and Goldwasser, M.H. (2014) Data Structures and Algorithms in Java. 6th edn. Hoboken, NJ: Wiley.

Hamill, P. (2004) Unit Test Frameworks: Tools for High-Quality Software Development. Sebastopol, CA: O’Reilly Media.

Pressman, R.S. and Maxim, B.R. (2020) Software Engineering: A Practitioner’s Approach. 9th edn. New York: McGraw-Hill.

Samadbeik, M., Saremian, M., Garavand, A., Hasanvandi, N., Sanaeinasab, S. and Tahmasebi, H. (2018) ‘Assessing the online outpatient booking system’, Shiraz E-Medical Journal, 19(4), e60249. doi: 10.5812/semj.60249.